مكتب

Homework Assignment

Week 4: String Basics

Total Points: 100 — Due: Next Class

Name: _____ **Date:** _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python *(using Thonny or any online IDE)*
- For setting up your code files, kindly see `instructions.pdf`
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: String Tracing Practice (15 points)

Time estimate: 20 minutes

1.1 Exercise 1.1: Trace the String Operations (8 points)

For this part you are not supposed to run the code on computer. Read this code carefully and trace through each step, filling in the following table:

```
message = "Hello"
result = ""
for i in range(4):
    result = result + message[i]
    print(f"Step {i+1}: {result}")
```

Fill in the trace table below:

Iteration	i	message[i]	result becomes	What prints
1	0	'H'	"H"	
2				
3				
4				

1.2 Exercise 1.2: Predict the Output (7 points)

Without running the code, predict what this prints:

```
text = "PYTHON"
print(text[0] + text[-1])
print(text[1:4])
print(text.lower()[2])
print(len(text))
```

Your prediction:

Line 1 output: _____

Line 2 output: _____

Line 3 output: _____

Line 4 output: _____

Now run the code to check your prediction. Were you correct?

2 Part 2: Code Completion - String Operations (18 points)

Time estimate: 25 minutes

2.1 Exercise 2.1: Username Validator (10 points)

Open week4_ex2_1.py and complete the username validation program. This will help you practice string methods and indexing.

```
# Username Validator
print("=== Username Checker ===")
username = input("Enter your username: ")

# Check if username is valid length (between 3 and 12 characters)
length = _______
if length < 3:
    print("Username too short! Must be at least 3 characters.")
elif length > 12:
    print("Username too long! Maximum 12 characters.")
else:
    print(f"Length OK: {length} characters")

# Check first character (must be a letter)
first_char = username[_______]
if first_char.isalpha():
    print(f"Good! Username starts with letter '{_______}'")
else:
    print(f"Error: Username must start with a letter, not '{_______}'")

# Check if username contains any spaces
if " " in _______:
    print("Error: Username cannot contain spaces!")
```

```
else:
    print("Good! No spaces found.")

# Display username in different formats
print("\nUsername formats:")
print(f"Uppercase: {_______}")  # Use upper() method
print(f"Lowercase: {_______}")  # Use lower() method
print(f"First 3 characters: {_______}")  # Use slicing
```

2.2 Exercise 2.2: Password Strength Checker (8 points)

Open week4_ex2_2.py and complete the password strength checker. This will help you practice string methods and searching.

```
# Password Strength Checker
print("=== Password Strength Checker ===")
password = input("Enter a password to check: ")

strength_score = 0

# Check 1: Length (at least 8 characters)
if len(_______) >= 8:
    strength_score = strength_score + 1
    print("Good length")
else:
    print("Too short (need 8+ characters)")

# Check 2: Contains uppercase letter
has_upper = False
for char in password:
    if char._______():  # Check if character is uppercase
        has_upper = True
        break
if has_upper:
    strength_score = strength_score + 1
    print("Has uppercase letter")
else:
    print("Missing uppercase letter")

# Check 3: Contains number
has_digit = False
for char in password:
    if char._______():  # Check if character is a digit
        has_digit = True
        break
if has_digit:
    strength_score = strength_score + 1
    print("Has number")
else:
    print("Missing number")

# Check 4: First and last characters are different
if password[_______] != password[_______]:  # Compare first and last
    strength_score = strength_score + 1
    print("First and last characters differ")
else:
    print("First and last characters are same")

```

```python
# Display strength rating
print(f"\nStrength Score: {strength_score}/4")
if strength_score == 4:
    print("STRONG password!")
elif strength_score >= 2:
    print("MEDIUM password - could be stronger")
else:
    print("WEAK password - please improve it")
```

3 Part 3: String Analysis and Comparison (15 points)

Time estimate: 20 minutes

3.1 Exercise 3.1: Compare String Operations (8 points)

Without running the code on a computer, study these three code snippets:

Code A:

```python
word = "HELLO"
result = word[0:3]
print(result)
```

Code B:

```python
word = "HELLO"
result = word[:3]
print(result)
```

Code C:

```python
word = "HELLO"
result = word[-3:]
print(result)
```

Fill in what each code prints:

Code A prints:	Code B prints:	Code C prints:
_____	_____	_____

Which slicing pattern would you use to:

- Get the first 5 characters of a name? _____
- Get the last 4 characters of a phone number? _____
- Copy an entire string? _____

3.2 Exercise 3.2: Fix the Logic (7 points)

This program should capitalize only the first letter of each word, but it's not working correctly:

```python
# Broken Title Formatter
text = "hello world python"
result = text.upper()  # This makes everything uppercase!
print(result)
```

Current output: HELLO WORLD PYTHON

Desired output: Hello World Python

Open week4_ex3_2.py and fix the code by using the correct string method:

```python
# Title Formatter
text = "hello world python"
result = _______._______()  # Use the right method
print(result)

# Test with another example
name = "ahmed ali khan"
formatted_name = _______._______()  # Same method
print(formatted_name)
```

4 Part 4: Problem Solving with Strings (18 points)

Time estimate: 25 minutes

4.1 Exercise 4.1: Message Encoder (18 points)

Scenario: Secret Message System

You're creating a simple message encoder that transforms text in special ways. The program should take a message and apply various transformations to create an encoded version.

Open week4_ex4_1.py and complete the message encoder program.

```python
# Secret Message Encoder

print("=== Secret Message Encoder ===")
message = input("Enter your secret message: ")

print("\nOriginal message:", message)
print("Message length:", _______(message), "characters")

# Encoding Step 1: Reverse the message
reversed_msg = message[_______]  # Use slicing to reverse
print(f"Step 1 - Reversed: {reversed_msg}")

# Encoding Step 2: Convert to uppercase
upper_msg = reversed_msg._______()
print(f"Step 2 - Uppercase: {upper_msg}")

# Encoding Step 3: Replace all spaces with asterisks
encoded_msg = upper_msg._______(_______, _______)
print(f"Step 3 - Encoded: {encoded_msg}")

```

```python
# Show first and last 3 characters of encoded message
if len(encoded_msg) >= 6:
    preview = encoded_msg[_______] + "..." + encoded_msg[_______]
    print(f"\nPreview: {preview}")
else:
    print(f"\nShort message: {encoded_msg}")

# Decoding verification - reverse the process
print("\n--- Decoding Test ---")
# Remove asterisks and add spaces back
decoded = encoded_msg.replace("*", " ")
# Convert to lowercase
decoded = decoded._______()
# Reverse again to get original
decoded = decoded[_______]
print(f"Decoded message: {decoded}")

# Check if decoding worked
if decoded.upper() == message.upper():
    print("Encoding/Decoding successful!")
else:
    print("Something went wrong!")
```

5 Part 5: Full Program Development (22 points)

Time estimate: 35 minutes

In this exercise, you will create a complete program that analyzes text and provides statistics.

Program Requirements

Your program should:

- Ask user to enter a sentence
- Count total characters (with and without spaces)
- Count how many vowels (a, e, i, o, u) are in the text
- Find the middle character(s) of the text
- Check if the sentence is a palindrome (reads same forwards and backwards)
- Display all statistics in a formatted report

Sample output:

```
=== Text Analyzer ===
Enter a sentence: A man a plan a canal Panama

--- Text Statistics ---
Original text: A man a plan a canal Panama
Length with spaces: 27 characters
Length without spaces: 21 characters

Vowel count:
```

```
Letter 'a' appears 10 times
Letter 'e' appears 0 times
Letter 'i' appears 0 times
Letter 'o' appears 0 times
Letter 'u' appears 0 times
Total vowels: 10

Middle character(s): a c

Palindrome check:
Forward: amanaplanacanalpanama
Backward: amanaplanacanalpanama
Is palindrome? YES!
```

Starter Structure:

Open week4_ex5.py and complete the text analyzer program.

```python
print("=== Text Analyzer ===")

# Step 1: Get user input
text = input("Enter a sentence: ")

print("\n--- Text Statistics ---")
print(f"Original text: {text}")

# Step 2: Calculate lengths
# find out the total length of the text:
total_length = ___________________________
no_spaces = text.replace(" ", "")
# Calculate length without spaces:
no_spaces_length = ___________________________

print(f"Length with spaces: {total_length} characters")
print(f"Length without spaces: {no_spaces_length} characters")

# Step 3: Count vowels
print("\nVowel count:")
text_lower = text.lower()
total_vowels = 0

# Count each vowel (a, e, i, o, u)
vowels = "aeiou"
for vowel in vowels:
    # Use count() method
    count = ____________________________
    print(f"Letter '{vowel}' appears {count} times")
    total_vowels = total_vowels + count

print(f"Total vowels: {total_vowels}")

# Step 4: Find middle character(s)
print("\nMiddle character(s): ", end="")
middle_index = len(text) // 2
if len(text) % 2 == 0:
    # Even length: show two middle characters, Use slicing
    middle_chars = ________________________________
else:
    # Odd length: show one middle character, Use indexing
```

```
    middle_chars = _______________________________
print(middle_chars)

# Step 5: Check if palindrome
print("\nPalindrome check:")
# Remove spaces and convert to lowercase for checking:
clean_text = _____________________________
# Reverse the cleaned text:
reversed_text = _____________________________

print(f"Forward: {clean_text}")
print(f"Backward: {reversed_text}")

if clean_text == reversed_text:
    print("Is palindrome? YES!")
else:
    print("Is palindrome? NO")
```

Testing Checklist:

- ☐ Program runs without errors
- ☐ Correctly counts characters with and without spaces
- ☐ Accurately counts all vowels
- ☐ Finds correct middle character(s)
- ☐ Palindrome check works properly
- ☐ Handles both uppercase and lowercase input

6 Part 6: Debug Detective (12 points)

Time estimate: 20 minutes

Each code snippet has issues. Find and fix them all:

6.1 Snippet 1: Name Formatter (4 points)

Open week4_ex6_1.py and fix the code that formats a name.

```
# Should print "Hello, AHMED!"
name = "ahmed"
greeting = "Hello, " + name.upper + "!"
print(greeting)
```

6.2 Snippet 2: String Slicer (4 points)

Open week4_ex6_2.py and fix the code that extracts parts of text.

```
# Should print first 3 and last 2 characters
word = "PYTHON"
first_part = word[3]  # Wrong! This gets one character
last_part = word[-2]  # Wrong! This gets one character
print(first_part + last_part)
```

6.3 Snippet 3: Character Counter (4 points)

Open week4_ex6_3.py and fix the code that counts characters.

```
# Should count how many times 'a' appears
sentence = "An apple a day keeps the doctor away"
count = 0
for char in sentence:
    if char = "a":  # Syntax error!
        count + 1    # Logic error!
print(f"Found 'a' {count} times")
```

7 Part 7: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. String indexing starts at 0, and the last character is at index length-1. Why is it important to remember this when working with strings?

2. When you use a string method like `upper()`, it returns a new string rather than changing the original. Why do you think Python works this way?

3. Which string operation did you find most useful in this homework: indexing, slicing, or methods? Give a specific example of when you would use it.

8 Bonus Section: Extra Challenges (Optional - 10 extra points)

Only attempt if you've finished everything else!

8.1 Challenge 1: Word Scrambler (5 points)

Create a program that scrambles words while keeping first and last letters in place:

- Input: "Python"
- Keep 'P' at start and 'n' at end
- Reverse the middle letters: "ytho" becomes "ohty"
- Output: "Pohtyn"

The program should work for any word with 3+ letters.

Save this code as week4_ex8_1.py

8.2 Challenge 2: Text Pattern Finder (5 points)

Create a program that finds all positions where a pattern appears in text:

- Ask user for a text string
- Ask user for a pattern to find
- Use a loop to check each possible position
- Report all positions where the pattern starts
- Example: Finding "an" in "banana" gives positions 1 and 3

Save this code as week4_ex8_2.py

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections
- ☐ Saved each code exercise in its own file
- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)
- ☐ Compressed all your code files into a single zip file named `week4_hw.zip`

Time Tracking:
How long did this homework take you? _____ hours

Parent/Guardian Signature: _____
(Confirming student completed work independently)

Great work mastering string basics!

Next week: String immutability and memory diagrams